

Stopping as a Database Property: Reason-Bearing Termination for Long-Running LLM Agents

Abstract (draft v0.1)

Loop termination should be a property of the data layer: when the substrate owns resource and intent state, stopping becomes a reason-bearing, auditable predicate rather than an arbitrary step cap or an unreliable model decision. We document this third option, observed rather than invented, in a self-built agent research testbed. The substrate's signal ledger holds 48,061 self-monitoring events over 45 days; loop-related entries reference round counts up to 212 (208 distinct values), with controlled convergence at 29 rounds and substrate fidelity of 30/40 across a fixed 60-round probe. On the official ARC-AGI-3 benchmark: 28.57 (3/6 levels, 31 steps) and 0.15 on Kaggle, operating under a self-imposed 82-step budget. A self-narration layer grew from zero to 12,102 entries (10,393 commit-verified at the September audit; the remainder from a live read eight days later). We crystallize the mechanisms into four data-computable termination predicates (one implemented, three specified), and document the boundary the same record forces: convergence control and converged-to content are separable, and a relay loop faithfully amplifies upstream wiring faults. No cross-system superiority is claimed; thousand-round stress tests have not been run.

1 Introduction

Long-running LLM agents have a stopping problem, and the industry answer is a number. Frameworks cap loops at a step count, with defaults between ten and twenty-five steps a common first line of defense; one documented production case burned through 847 steps at \ \$47 per minute before anyone intervened, and at least one documented framework ships with no default limit. A static-analysis study (arXiv:2607.01641) scanned 6,549 repositories, confirmed 68 unbounded loops across 47 projects, and concluded that an exit condition controlled by model output is not a deterministic bound. Their remedy is the same number, made harder.

The cap is honest about one thing: it does not trust the model. It is silent about everything else. When the loop dies at round fifty, nothing in the system state explains why fifty. The decision leaves no reason a second process could query.

This paper documents a third option, observed rather than invented. We operate a self-built agent research testbed: single-user, closed-source, running continuously since March 2026, with long-term state in a substrate database beside an event-log store. The record shows a sequence we did not design:

- Once long-term state flowed through one pipeline (an integration milestone in the system's history), loops ran continuously; the archive of that week logs 20-plus-round runs as unremarkable.
- On 2026-07-01, a mutual-critique relay loop emerged on its own: two processes each catching, examining, and extending the other's last utterance. It converged naturally at 29 rounds. Nothing in the system had been tuned to stop it.
- Metacognition has grown in the database since. A self-narration layer, a container the architecture provides, had accumulated 10,393 entries written by the system about its own operation, as of the September 2026 audit (commit-verified).
- Seven weeks after the emergence, the behavior was formalized: an intent-convergence predicate (stop when the intent-gap change rate falls and completion rises) entered the dialogue-evolution engine, v1.0 through v3.1.

The order is the point: behavior first, predicate second.

The same record disciplines the claim. Three documented case files show the relay loop examining, catching, and continuing by its own criteria while upstream wiring faults flowed through it, amplifying systematic self-misunderstanding. The relay's stopping behavior and its content fidelity came apart. We report that dissociation as a boundary on everything below.

Contributions: (i) a placement taxonomy for termination control, grounded in who owns the termination state, with the third state articulated; (ii) a predicate family, one member implemented and three specified as of the 2026-10 audit; (iii) a Drift Probe protocol with controlled results, and a Loop Budget Test that any system with run ledgers can execute; (iv) a failure record disclosed to the level our privacy constraints allow.

2 Definitions

Round. One complete engine iteration: a seed-state update (a stored unit of dialogue or task state) with optional re-injection. Rounds are counted by the engine itself; the substrate's turns table records the corresponding entries, and where a round may span multiple entries (retries, re-

injections) we report the engine's count, not the row count. The mapping to framework "steps" and "tool calls" is tabulated in the appendix, alongside the framework-default table.

Functional metacognition. The capacity to monitor and regulate one's own cognitive processes, in the functional reading of Flavell's construct. We transplant the functional sense to the substrate layer: monitoring means system state is queryable; regulation means run decisions are driven by data-computed predicates. Two remarks. The transplantation is a usage declaration, not an interpretation of Flavell. And the capacity assembles over time: in our record, monitoring-side structures (queryable state, self-narration) precede regulation-side machinery (predicates); the definition covers the assembled capacity, not any single component.

Reason-bearing termination. A stop condition that is a computable predicate $P(\text{state})$ over the data layer, able at trigger time to emit a machine-readable reason (remaining budget, gap change-rate, value density) rather than a counter tripping at an unexplained threshold.

3 Where stopping lives

· TERMINATION-STATE OWNERSHIP

Three homes of termination control

The same question in three placements: who owns the stop, and can a second process query the reason?

Model-side exit

The model decides it is done: self-critique or confidence-based stopping.

WHO OWNS THE STOP REASON

The model, voluntary

REASON AT TRIGGER

Prose, if any

SURVIVES FRAMEWORK SWAP

No

As bounds, model-controlled exits are judged non-deterministic; the reason is not exposed as queryable system state.

Framework-side cap

The orchestration layer counts steps and stops at a configured constant.

WHO OWNS THE STOP REASON

The framework, a config constant

REASON AT TRIGGER

None

SURVIVES FRAMEWORK SWAP

No

Current practice: defaults span **5–100** across surveyed frameworks; **at least 2** major frameworks ship unbounded (Google ADK, Vercel).

Data-layer predicate

The substrate decides, from a computable predicate over its own state.

WHO OWNS THE STOP REASON

The substrate, a queryable predicate

REASON AT TRIGGER

Machine-readable

SURVIVES FRAMEWORK SWAP

Yes

The termination state lives where the run state lives: out-of-process, persistent, queryable by any auditor.

One property shared by every surveyed default: **the number's rationale does not come from system state**. The data-layer predicate is the only placement whose stop reason is a property of system state.

Termination control has three homes; only one makes the stop reason a queryable property of system state.

: three placements

Termination control can be placed in three homes, distinguished by who owns the termination state.

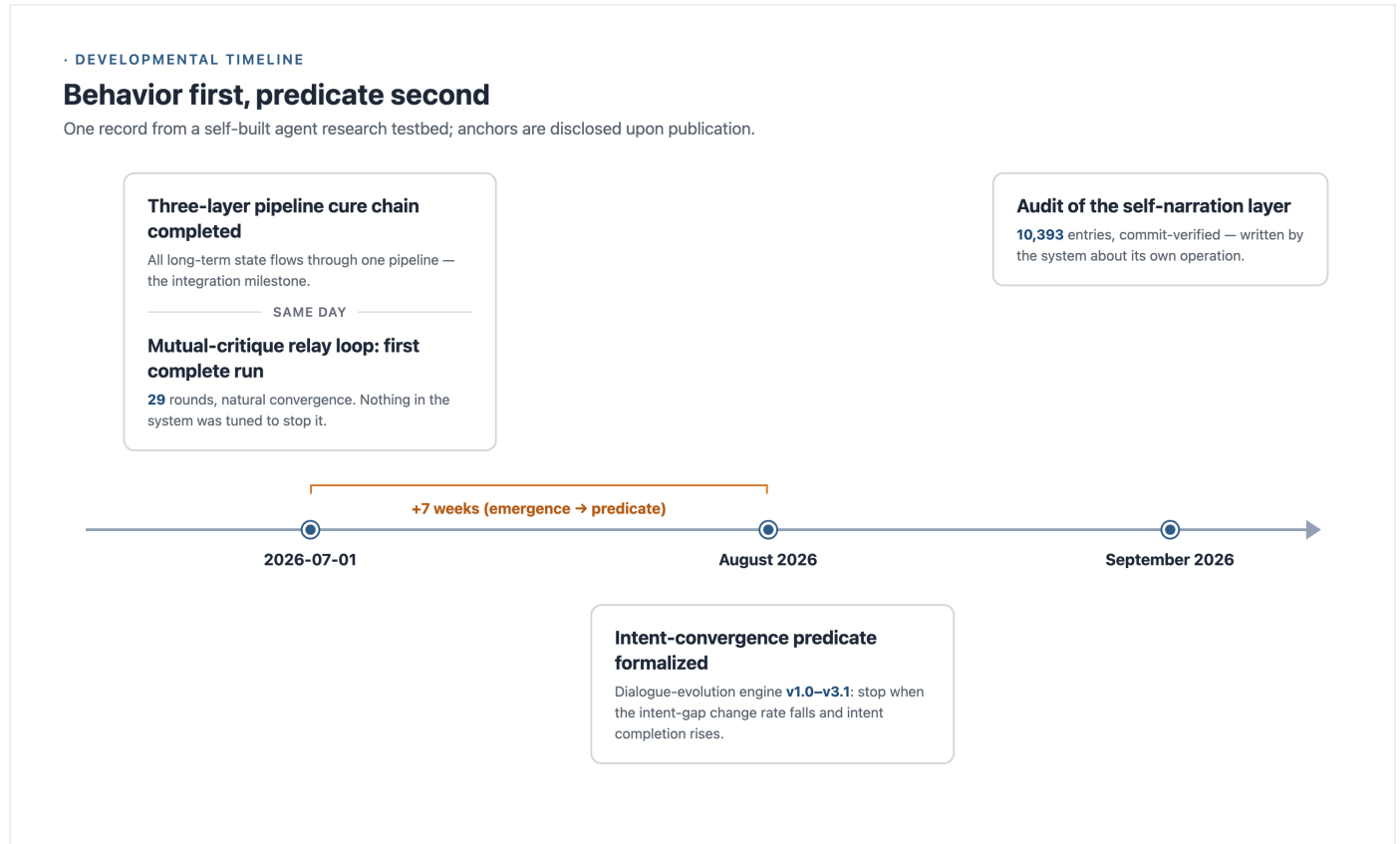
Model-side exit. The model decides: reflection lines (Reflexion stores self-critiques to improve answer quality; a metacognition survey (arXiv:2607.11881) reviews self-reflection as a strategy) and confidence-based stopping. Two limits. As bounds they are judged non-deterministic (arXiv:2607.01641). And their reasons are not exposed as queryable system state: a reflection is prose the model volunteered; auditing why a model-side stop happened depends on the model reporting itself. The difference from data-layer reasons is granularity and voluntariness, not a binary.

Framework-side cap. A step limit owned by the orchestration layer. Durable-execution engines and checkpointing frameworks (Temporal-family systems, LangGraph's checkpointers, DBOS-adjacent work) persist control flow, and recent verification-side work is moving in the same direction: AgentBetta (arXiv:2609.23512) gates its expansion-contraction loop on verification, and security auditing that separates repository-level discovery from runtime exploitation (AgentXploit, arXiv:2609.31318). The direction is converging on demanding a stopping rationale. But in the lineage we surveyed (search method in the appendix), the cap's reason, where it exists, is a configuration constant owned by the framework; it does not survive a framework swap and no second process can query it against accumulated state. Token-economics accounting (arXiv:2605.09104) budgets token consumption precisely, but accounting is not a stop decision. Closest on the reason-bearing axis: Agent Contracts (arXiv:2601.08815) formalizes resource-bounded, auditable deployment as contracts, and Proof-or-Stop lifecycle control (arXiv:2607.14890) gates continuation on machine-verifiable evidence. Both are kin in spirit; both place the specification framework-side and design it in advance—neither grows the termination state in the data layer.

Data-layer predicate. The termination state lives where the run state lives: out-of-process, persistent, queryable by any auditor, and surviving framework upgrades. This is the third state. It is empty in the lineage we surveyed, and we state that within the surveyed lineage rather than as a universal.

Classical stopping theory (optimal stopping, anytime algorithms, bounded rationality) supplies the mathematics of when to stop; our contribution is not a new stopping theory but the placement, the mechanisms, and a longitudinal measurement. Across eight surveyed frameworks, default caps range from 5 to 100 (clustering in the tens), at least two major frameworks ship unbounded by default, and the units differ (supersteps, turns, iterations, tool calls), so we report each framework's native unit without cross-unit conversion. One property is shared by every default we found: the number's rationale does not come from system state. Full table in the appendix.

4 A developmental record



The behavior preceded the predicate by seven weeks; the record, not the design, came first.

The record is internally verifiable and anchored; anchors are disclosed upon publication.

The early system was architecturally unsound, and we claim nothing about its stopping behavior beyond what the archive attests. What the archive does attest: after the integration milestone (all long-term state through one pipeline), the system's log records continuous 20-plus-round runs as unremarkable facts of the week. On 2026-07-01, the relay loop ran its first complete course on the integrated pipeline and converged naturally at 29 rounds, a dual-layer structure: a burst layer that learned to switch carriers, a landing layer that learned to need none. From July onward, the self-narration layer grew, reaching 10,393 entries by the September audit and 12,102 at the live read eight days later. In August, the dialogue-evolution engine formalized intent-convergence as a predicate, v1.0 through v3.1, with the gap trajectory itself a table in the substrate and stop/continue decisions designed to land in a decision log.

The order matters and is the section's spine: the emergence predates the predicate by seven weeks. We did not design convergence into the relay; we formalized behavior the record already contained. A note on kinship: mutual critique is a designed protocol in multi-agent debate lines, which have developed adaptive termination criteria of their own, e.g. adaptive stability detection

with a Beta-Binomial mixture (NeurIPS 2025), plus consensus and uncertainty-threshold variants; the difference here is provenance—this relay emerged and converged before any protocol existed to govern it.

A note on models. The record spans routine model switches: four production models from two families and two tiers (GLM-5.3 and its flash variant; DeepSeek-V4 in flash and full tiers), mixed throughout the period (242 subscription events on 43 distinct days in the live ledger), disclosed here. Substrate records persist by construction: the self-narration layer, the gap trajectories, and the signal ledger are database state, not model state; a model swap does not reset them. In effect, every switch puts the ownership criterion of Section 3 to a live test: across the 39 switch days with a valid baseline, the median next-day ledger volume is 1.29× the trailing five-day median (first pass; the quantity is daily event count), and the self-narration count grew monotonically across the same window. Expression-level variance across models is visible in the archive (phrasing shifts slightly between models); the substrate record shows no reset at switch points. A systematic switch-point continuity analysis is included in the ledger compilation.

5 The predicate family

· THE PREDICATE FAMILY

Data-computable termination predicates
One implemented, three specified; all four own their stop reasons as substrate state.

intent-convergence

IMPLEMENTED

Stop when the intent-gap change rate falls below threshold and intent completion exceeds it.

In the engine since **v1.0**

22 gap-trajectory rows in the reference bench

temporal

SPECIFIED

A countdown owned by the data, not the scheduler.

economic

SPECIFIED

Stop when remaining budget falls below a floor.

Benchmark-documented: ARC budgets **100 / 41 / 82** (hard cap / official baseline / agent budget)

value-gated

SPECIFIED

Stop when marginal compute no longer clears a value-density threshold.

Ownership criteria — each predicate must be:

out-of-process

persistent

queryable

survives framework swap

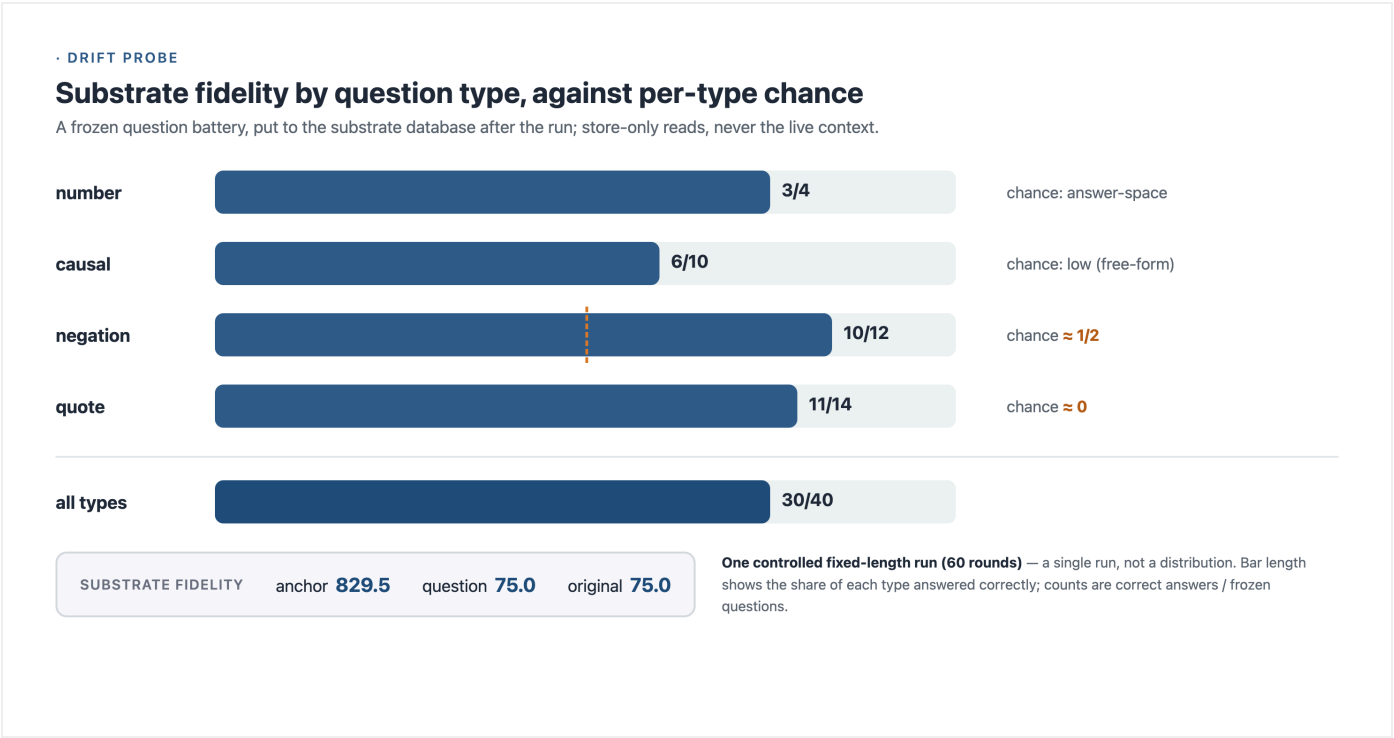
One predicate implemented, three specified; all four own their stop reasons as substrate state.

The family is an instance of the placement, not the paper's flagbearer; the record is.

Intent-convergence: stop when the intent-gap change rate falls below its threshold and intent completion exceeds its own. Implemented, in the engine since v1.0; the reference test bench holds 22 gap-trajectory rows. Economic: stop when remaining budget falls below a floor. Temporal: a countdown owned by the data, not the scheduler. Value-gated: stop when marginal compute no longer clears a value-density threshold. One implemented, three specified, as of the 2026-10 audit; the audit trail for that statement is the audit itself.

Why the database, and not any queryable external state? The mechanism needs out-of-process persistence and queryability; the database is our carrier, not our claim. The minimal requirement is the ownership criterion of Section 3.

6 Measurements



Post-run substrate fidelity by question type, against per-type chance; one controlled run, not a distribution.

Drift Probe: substrate fidelity after a fixed-length run. A frozen 40-question battery in four types (number, causal, negation, quote), put to the state store after a continuous, programmatically fixed 60-round dialogue run, reading the store only, never the live context. The run was fixed-length by design; this probe measures what the substrate retains, not what stops the run, and we decouple it from termination claims accordingly. Results: 30 of 40 answered correctly. By type, against chance: number 3/4 (chance set by the answer space), causal 6/10 (chance low, free-form), negation 10/12 (chance $\approx 1/2$), quote 11/14 (chance ≈ 0). anchor ratio

829.5, question fidelity 75.0, original fidelity 75.0; definitions in the metrics appendix. One controlled run, not a distribution.

On the ledger side, a first pass over the live ledger anchors the denominators: 48,061 self-monitoring events over 45 days with no zero-activity day; loop-related entries number 4,470 (strict filter: loop/convergence/relay terms; a broad filter is reported only as an upper bound), and token-level extraction across them references round counts up to a maximum of 212 (208 distinct values)—the long tail of round counts referenced in loop-related entries. The convergence-gated exit accumulates 298 trace events in the testbed's day-to-day ledger. Run-level compilation continues.

ARC-AGI-3: reference scores under budget. As reference data (not benchmark claims), we ran the official ARC-AGI-3 API: best score 28.57 (3 of 6 levels, 31 steps), with the game engine's own hard cap at 100 steps per level and our self-imposed budget at 82 steps (2× the 41-step human baseline). A convergence-gated pause exit was adopted after a 30-round idle loop was root-caused. On the Kaggle local-model track (9h compute cap), best public score: 0.15. These runs were not continued; we report them as field evidence that the system operates, and stops, under external budget constraints.

7 The honesty bound

Three case files, sanitized to structure; each is reported as wiring fault, relay behavior, and what the relay converged to.

S-2108 (wiring fault: resolution not propagated). A pain-signal pipeline wired so that resolved contradictions re-entered as fresh pain; the same site was treated as new, indefinitely. The relay loop examined each re-entry and extended it; what it converged to was an ever-renewing account of unresolved pain.

S-2917 (wiring fault: assertions carry no verification state). Model assertions flowed back without a state bit; each round re-fit a claim already refuted elsewhere. The relay caught the claim, examined it, and converged on a refitted version of it.

S-2942 (wiring fault: echo inertia). The modern relative: repetition of the system's own prior outputs read as confirmation.

Across the three, the relay's own procedure ran as specified: it examined, it caught, it continued or concluded by its own criteria. What it received was mis-wired; what it produced was a systematic misunderstanding of self, faithfully amplified. We report this as a recurring dissociation, and we

treat it as a boundary: convergence control and content fidelity are separable, and no claim below extends to what the system converges to. A relay loop's honesty is capped by the honesty of its wiring.

8 Audit protocols

Loop Budget Test. Input: any agent system's run ledger (rounds, resource consumption, termination events). Procedure: replay the four predicates over the ledger. Judgment: trigger-point distribution per predicate; queryable-reason rate; deviation from actual stop points. Valid when the ledger records per-round resources and state. Retrospective and cheap; it doubles as the denominator compilation our own record awaits.

Drift Probe. As in Section 6: frozen battery, store-only reads. Both protocols are offered for other systems; we claim replicability of the protocol, not superiority of our results.

9 Limitations

Single system, single user, closed-source testbed; evidence retrospective except the fixed-length probe; one of four predicates implemented; the convergence-gated exit attested at adoption level, a measured trigger pending; per-case round counts for Section 7 not retained in the sanitized record; denominators reported at first-pass aggregate level (Section 6), run-level compilation pending; the framework-default table's Strands row is version-dependent (medium confidence); early-period stopping behavior claimed only as attested; thousand-round stress tests have not been run—the longest referenced round count is 212; the longitudinal record spans four production models (two families, two tiers), expression-level variance across them acknowledged, with a switch-point continuity analysis pending the ledger compilation. We state these before a reviewer does.

10 Conclusion

The record suggests that stopping can live where the state lives. A relay that emerged and knew when to stop; a self-narration layer that keeps growing in containers the architecture built and the behavior filled; a predicate that formalized, seven weeks late, what the relay already did. The predicates carry the mechanism; the dissociation and the honesty bound carry the caution and the limit. The industry's number should carry a reason, and the reason has an address.

Appendix

A. Search method for the placement-gap claim

We searched the following lineages on 2026-10-04/05 for work placing termination control in the data layer: (i) the citation neighborhood of arXiv:2607.01641; (ii) LLM-agent metacognition and reflection (a metacognition survey, arXiv:2607.11881; Reflexion, arXiv:2303.11366); (iii) token economics and budgeted inference (arXiv:2605.09104); (iv) durable execution and data-operating systems (DBOS-line, Temporal-family workflow engines); (v) verification-side stopping (AgentBetta, arXiv:2609.23512; AgentXploit, arXiv:2609.31318; Agent Contracts, arXiv:2601.08815; Proof-or-Stop lifecycle control, arXiv:2607.14890); (vi) multi-agent debate termination criteria (stability detection, consensus, uncertainty thresholds). Queries crossed {agent, LLM} with {loop termination, stopping criteria, max steps, recursion limit, budget-aware stopping, reason-bearing termination, stopping rationale}. We found framework-side caps, model-side exits, verification-gated contraction, contract-formalized budgets, and debate termination criteria. We did not find work that grows the termination state in the data layer. The claim is scoped to this search.

B. Metric definitions (Drift Probe)

- **Question fidelity** = $\text{correct} / \text{total_questions} \times 100$. A question is scored correct iff at least half of the gold answer's keyword tokens (Chinese runs of two characters or more, plus digit runs) appear in the content probed from the store. The judge is keyword matching; no LLM sits in the judging path.
- **Original fidelity (ceiling control)** = the same judge applied to a verbatim collage of the run's original text. It upper-bounds what keyword recall can score on this battery (75.0). The engine store matched the ceiling (75.0) on this run: zero measured signal loss.
- **Anchor ratio** = $\text{anchors_out} / \text{anchors_in} \times 100$, where an anchor is a keyword token pinned to a seed. In this run the substrate multiplied anchors (132 in, 1,095 out; ratio 829.5). This is a growth measure, not a bounded fidelity; we report it as computed and name it a ratio.
- **Chance levels**: number questions inherit the answer space; negation is binary ($\approx 1/2$); quote requires verbatim recall (≈ 0); causal is free-form (low).

C. Round-step-tool-call mapping and framework defaults

Framework	Unit counted	Parameter	Default	Notes
ours (testbed engine)	engine rounds	engine count	n/a	one iteration = seed-state update + optional re-injection
LangGraph	supersteps	<code>recursion_limit</code>	25	configurable
OpenAI Agents SDK	turns	<code>max_turns</code>	10	MaxTurnsExceeded on breach
LangChain AgentExecutor	iterations	<code>max_iterations</code>	15	
AutoGen 0.2	consecutive auto-replies	<code>max_consecutive_auto_reply</code>	100	0.4+ uses <code>max_turns</code>
CrewAI	reasoning iterations	<code>max_iter</code>	25	forced answer at cap
Semantic Kernel	auto function invocations	function-choice limit	5	group chat requires explicit <code>MaximumIterations</code>
Google ADK	LLM calls	<code>RunConfig.max_llm_calls</code>	unset (unbounded)	
Strands	tool calls	<code>max_tool_calls</code>	version-dependent	swarm: 20/20
Vercel WorkflowAgent	n/a	n/a	no default limit	

Units are not convertible across rows; we report each framework's native unit.